MiniDB · файловая мини-СУБД

Устройство, хранение и команды

Пример создаёт ./sample.sh: скрипт передаёт SQL в run.sh. Все три таблицы создаются сразу через CREATE TABLE; ADD COLUMN проверяется отдельно. INSERT добавляет 5/5/20 записей. Открытие: ./run.sh sample. Rust-модуля со схемами примера нет.

Схема данных

datasets

id INT · PK

name STRING

task STRING

size INT ≥ 0

experiments

id INT · PK

dataset_id INT · FK

model_id INT · FK

accuracy FLOAT [0,1]

status STRING

models

id INT · PK

name STRING

version STRING

experiments.dataset_id → datasets.id; experiments.model_id → models.id. Связи N:1, RESTRICT при удалении и изменении ключа родителя. Все поля примера NOT NULL; при INSERT в datasets обязательны name, task и size. DBML для dbdiagram: docs/schema.dbml.

Читаемые файлы данных и текстовые схемы

Ветка text хранит рабочие данные в UTF-8 без расширений: sample/datasets, sample/models, sample/experiments. Схема остаётся в имя.schema.sql. В файле данных — VERSION, имена полей, LIVE/DEAD/FREE и значения через Tab. Строки в кавычках, управляющие символы экранируются. Это основное хранилище; бинарных копий рядом нет. Следующие версии находятся в vN/имя. Для ручной правки сначала закройте CLI. Бинарная реализация сохранена в ветке binary.

Сегменты загружаются в RAM; Layout и Row — общее представление внутри программы. Обычная запись сохраняет UTF-8 snapshot через временный файл, sync и rename. Seed/bench отключают барьеры отдельных записей до финального sync. Auto-ID восстанавливается из LIVE/DEAD и checkpoint в схеме. Backup копирует схемы и все сегменты; RESTORE может заменить текущую базу. Между несколькими сегментами транзакционного отката нет.

Примеры команд внутри MiniDB

```
CREATE TABLE notes (id INT PRIMARY KEY, name STRING);
INSERT INTO notes VALUES (NULL, 'Alice');
SELECT name FROM notes WHERE id = 1;
UPDATE notes SET name = 'Ann' WHERE id = 1;
DELETE FROM notes WHERE id = 1;
backup 'data/copy'
restore 'data/copy' to 'data/restored'
```

Также: проекция, шесть сравнений, AND/OR со скобками, IS NULL и INNER JOIN INT-поля левой таблицы с PK правой. Изменения сохраняются после каждой успешной команды.

MiniDB · файловая мини-СУБД

Проверка, benchmark и ограничения

Индекс: полный просмотр и точечное чтение

Текст декодируется при открытии. В RAM находятся строки и HashMap id → (версия, смещение в памяти). Scan использует рабочий буфер; at() декодирует поле, а ограничения проверяются отдельно. Benchmark сравнивает полный просмотр и индекс в RAM; это не тест случайного доступа к диску и не сравнение с binary.

Release, 5000 записей, 1000 одинаковых детерминированных запросов; данные уже в RAM. Генерация, commit/fsync и открытие исключены из измерения поиска. Контрольные суммы результатов совпали: 2486500.

Полный просмотр	65.834 мс
Индекс id → смещение	0.158 мс
Отношение времени	415.75×

Это один локальный замер, не гарантия скорости на другой машине. Воспроизведение: cargo run --release --example bench -- scratch/bench 5000. Для другого числа записей нужен новый каталог.

Проверенное поведение

Автотесты: CRUD и рестарт; фильтры и NULL; PK/auto-ID, FK, RESTRICT и JOIN; backup/restore и блокировка; числа, STRING и UTF-8; консоль, sample.sh, эволюция схем и перенос старого формата. SHOW DATABASE показывает абсолютный путь активной базы.

Проверены UTF-8 и escapes, повторное открытие после правки в IDE, повреждённые заголовки и поля, коллизия временного файла, backup всех версий и отказ restore без потери прежних данных. Тесты бинарных слотов остались в binary.

Инженерное расширение

SQL-подобный парсер, консоль и Docker с постоянным томом. Контейнер собирает и проверяет код, запускает консоль непривилегированным пользователем. Запуск: docker compose run --build --rm minidb. Инструкции запуска sample.sh внутри контейнера есть в README.

Границы реализации

Индекс требует O(числа живых записей) памяти. До 256 полей; STRING по умолчанию 64 байта, максимальная ёмкость и длина — 65535 байт. Пакет UPDATE/DELETE ограничен 16 МиБ. Нет составных PK, self JOIN, vacuum и многооператорных транзакций.

ALTER ADD добавляет nullable-поле в конец. Без типа оно остаётся NULL до первого ненулевого INSERT/UPDATE; выбранный тип сохраняется. Исходные поля не переопределяются. Генератор создаёт Rust-код из схемы; потребитель компилирует его своим build.rs. Типизированная запись проверяет VERSION. Перенос UPDATE между сегментами не транзакционен. В seed до sync записи находятся только в памяти. Физическое отключение питания не тестировалось.

Материалы: README.md, sample.sh, docs/COMMANDS.md, docs/schema.dbml, docs/benchmark.csv.